

方法一，在容器中编译go代码

```
1 # 指定基础镜像，必须存在于Dockerfile 第一行
2 FROM daocloud.io/golang
3
4
5 # 作者信息
6 MAINTAINER wolferhua <wolferhua@qq.com>
7
8 # 添加文件到镜像
9 ADD . /usr/src/app
10
11 # 切换目录，相当于cd xxx
12 WORKDIR /usr/src/app
13
14 # 执行编译g
15 RUN go build -o app main.go
16
17 # 切换目录
18 WORKDIR /app
19
20 # 将编译后的移动到 /app 目录
21 RUN mv /usr/src/app/app .
22
23 # 开放端口，在运行时，并不会正在的开放端口。还是需要-p
24 # 这里只是标明需要使用的端口
25 EXPOSE 8080
26
27 # 镜像启动时执行的命令
28
29 CMD ["/app"]
30
31 # 构建命令
32 #docker build -t goweb .
```

这种方式有以下几个缺点：

- 1, 编译的image过大
- 2, 如果有包依赖还得下载包依赖, 这一步容易出错, 所以一般不用这种方式

第二种方式, 在外部编译好Go的二进制包, 然后添加到image中

```
1 # 指定基础镜像, 必须存在于Dockerfile 第一行
2 # 使用基础镜像
3 FROM alpine:latest
4
5 # 作者信息
6 MAINTAINER wolferhua <wolferhua@qq.com>
7
8 # 切换目录, 相当于cd xxx
9 WORKDIR /app/
10
11 # 添加文件到镜像
12 ADD /main /app/
13
14 # 执行命令
15 CMD ["/app/main"]
```

因为在基础镜像中没有其他内容, 所以要执行以下命令, 使生成的main静态链接所有的库

```
1 CGO_ENABLED=0 GOOS=linux go build -v -a -installsuffix cgo -o main .
```

几个基础镜像的比较:

scratch: 这个镜像是一个空的镜像, 没有任何内容。且不能执行任何命令 不般不使用

busybox: 是一个集成了一百多个最常用Linux命令和工具 (如cat、echo、grep、mount、telnet等) 的
精简工具箱, 它只有几MB的大小, 很方便进行各种快速验证, 被誉为“Linux系统的瑞士军刀”。

BusyBox可运行于多款POSIX环境的操作系统中, 如Linux (包括Android)、Hurd、FreeBSD等

Alpine操作系统是一个面向安全的轻型Linux发行版。它不同于通常的Linux发行版, Alpine采用了musl libc和BusyBox以减小系统的体积和运行时资源消耗, 但功能上比BusyBox又完善得多。在保持瘦身的同时, Alpine还提供了自己的包管理工具apk, 可以通过<https://pkgs.alpinelinux.org/packages>查询包信息, 也可以通过apk命令直接查询和安装各种软件。

alpine Docker镜像也继承了Alpine Linux发行版的这些优势。相比于其他Docker镜像，它的容量非常小，仅仅只有5MB左右（Ubuntu系列镜像接近200MB），且拥有非常友好的包管理机制。官方镜像来自docker-alpine项目。

所以在docker image中如果没有其他特别要求，推荐使用：**alpine**